Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

In flexible job shop scheduling that captures processing order with an operation-precedence graph and records the corridor occupancies of vehicles in a time-window reservation table, the rescheduling scope after a disturbance is still delineated by affected-operation rescheduling (AOR): operations that can no longer be executed as planned are identified, and their successors on the operation-precedence graph are added. Corridor blockage or slowdown acts only on corridors; machines and operations remain available, so AOR finds no affected operation and returns an empty scope, concluding that no rescheduling is needed. Yet the planned occupancies inside the disturbed interval are already invalid and the original schedule is infeasible; moreover, a corridor admits one vehicle at a time, so a vehicle planned to traverse it later must wait for the preceding one (a yielding relation), and a delay passes from vehicle to vehicle. The problem is not detection (the blockage is known) but that the operation-precedence graph used by AOR contains only operations and their precedences, with neither corridor occupancies nor the yielding relations among them.

We instead delineate the rescheduling scope at the level of corridor occupancies and propose the spatiotemporal reservation closure (STRC), whose output set of occupancies is called the reservation impact set. Starting from the planned occupancies whose intervals overlap the disturbed interval, STRC keeps adding successor occupancies along four dependency relations among occupancies—yielding on the same corridor, and the next occupancy of the same vehicle, of the same job, and on the same machine—until none can be added (the transitive closure). Each occupancy is determined by its corridor (path), interval, and vehicle: occupancies inside the set may have their path and interval replanned (and, under vehicle failure, their vehicle replaced), while those outside keep all three. Once the four relations are fixed, membership in the set depends only on reachability from the seeds and is independent of the rerouting algorithm. Bounded repair first replans conflict-free paths for the occupancies inside the set with the original vehicles; if the schedule remains infeasible, it successively adds the subsequent occupancies of the same job, of the same vehicle, and finally all unfinished occupancies to the rewritable scope.

On 50 instance–seed pairs (5 instances × 10 random seeds that determine disturbance injection and search), the scope delineated by AOR under corridor blockage is empty on all pairs, although the original schedule is in fact infeasible on every one of them; with the rerouting procedure unchanged and only the scope object replaced, the number of pairs repaired to a feasible schedule rises from 0/50 to 50/50. On all pairs the impact set is closed (no dependency edge leads from inside to outside), and every occupancy outside it keeps its original corridor, interval, and vehicle after repair. Rewriting all unfinished occupancies instead leaves runtime and makespan almost unchanged but raises the fraction of rewritten occupancies from 0.540 to 0.604 (scoping by the impact set rewrites fewer/equally many/more on 31/18/1 pairs; Wilcoxon $p = 8.8 \times 10^{-7}$). Compared with warm-started global rescheduling that re-searches assignment and order within a time limit, bounded repair is two to three orders of magnitude faster, but its makespan is never better under any tested time limit and cannot overtake under longer limits. Feasibility is thus restored by the scope object; the reduction in rewrites is

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

limited because the impact set already covers about 92% of all unfinished occupancies. Rerouting inside the set and enlarging the scope after failure follow existing practice; the contribution lies in delineating the rescheduling scope on corridor occupancies instead of operations.

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

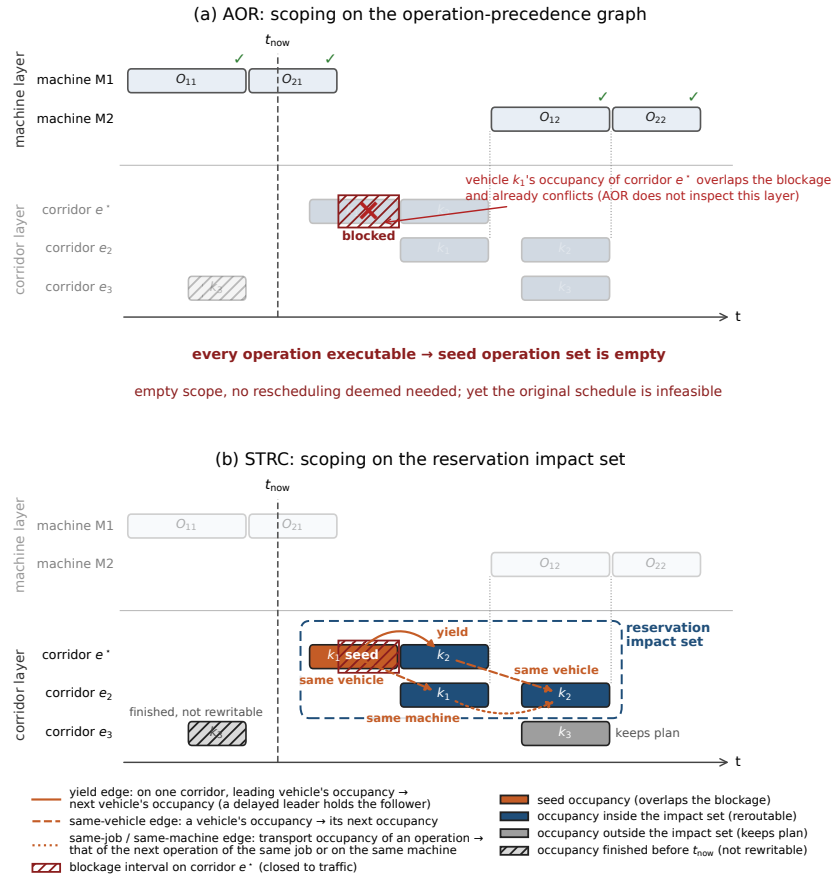
Additional Key Words and Phrases: flexible job shop scheduling, bounded rescheduling, conflict-free AGV routing, spatiotemporal reservation table, spatiotemporal reservation closure, corridor blockage

ACM Reference Format:
Qi Yu and Wei Li. 2026. Bounded Rescheduling for Flexible Job Shops via Spatiotemporal Reservation Closure. 1, 1 (November 2026), 22 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Flexible job shop scheduling with transportation usually takes transport times as constants indexed by location pairs [13, 16, 25], assuming no corridor contention among vehicles. When a finite fleet shares a sparse corridor network this no longer holds [5], so existing work decides machine assignment jointly with conflict-free paths before execution [7, 9, 20, 23], and a schedule then also contains a time-window reservation table recording which vehicle occupies which corridor during which interval. A disturbance may make operations unexecutable (e.g., machine or vehicle failure) or invalidate only some corridor occupancies without changing any operation (e.g., a corridor blocked or slowed down for some period). These works answer how to build a conflict-free schedule when information is complete. Once the shop has executed up to time t_{now} and a disturbance occurs, the questions become: which corridor occupancies are invalidated; which must be rewritten so that the schedule again satisfies corridor exclusivity and the closure constraints and can continue to be executed; and whether such a schedule can be obtained within milliseconds of computation time without restarting a population-based search.

The gap. In dynamic scheduling of flexible job shops with transportation, the rescheduling scope is usually delineated on the operation-precedence graph: operations that can no longer be executed as planned serve as seeds, the operations following them in the same job and on the same machine are added, possibly only up to θ steps (a manually set upper bound on the expansion depth), and only the operations in this scope are rescheduled. Existing dynamic scheduling studies with AGV transport delineate the scope in this way [22, 26], a practice represented by affected-operation rescheduling (AOR) and its modification MAOR [18]. This is reasonable when the disturbance is an operation event, such as a machine failure. When the schedule records corridor occupancies with time windows, however, a *corridor blockage* closes a corridor during $[t_a, t_b)$ without modifying any machine assignment or processing order; machines and operations remain available. The seed set of AOR (operations that can no longer be executed as planned) is therefore empty, so the scope is empty and the method concludes that no rescheduling is needed. Yet the planned occupancies that overlap the blockage interval are no longer feasible. Moreover, a corridor admits one vehicle at a time, so a vehicle planned to traverse it later must wait for the preceding one to leave (a yielding relation); a vehicle delayed by waiting or detouring therefore delays its follower, which delays its own follower, and the invalidation propagates across the reservation table. The blockage itself is known, so the problem is not detection: the operation-precedence graph used by AOR has operations as vertices and job and machine precedences as edges, and by definition contains neither corridor occupancies nor their yielding relations, so it cannot express this invalidation or its propagation; the gap arises only in settings that record corridor occupancies with time windows. We accordingly distinguish **Class A** disturbances, which



Notation: O_{ji} is operation i of job j; M1, M2 are machines; k_1, k_2, k_3 are vehicles (k_1 carries job 1, k_2 carries job 2, k_3 carries another job); $e^\star$, e_2, e_3 are corridors; each bar in the corridor layer is one occupancy of a corridor by a vehicle, and a thin dotted line links it to the operation it delivers to.

Fig. 1. Motivating example: the same schedule under the same blockage on corridor $e^\star$ (hatched), with machine layer above and corridor layer below. (a) AOR inspects only the machine layer: every operation is executable, so the scope is empty, although vehicle k_1's occupancy already conflicts with the blockage. (b) The proposed method inspects the corridor layer: vehicle k_1's overlapping occupancy is the seed, and yield, same-vehicle, and same-machine edges add subsequent occupancies to the reservation impact set (dashed box); occupancies outside it keep the original plan.

make some operations unexecutable (robotic-arm failure, processing-time deviation, rush orders, vehicle failure), from **Class B** disturbances, which act only on the reservation table (corridor blockage, corridor slowdown); under Class B, scoping on the operation-precedence graph returns an empty scope (Section 3.3). Figure 1 illustrates the gap.

Stance. The reservation impact set answers only *which occupancies are already invalid*, not *how the schedule should be changed*. We use the dependencies only to decide invalidity: an occupancy reachable from the seeds belongs to the impact set, one that is not does not. These relations are not used to steer the search or to change assignments and paths; that is a decision problem of schedule improvement outside our scope. The impact set is not necessarily the minimal set

needed to restore feasibility, because the minimal set depends on the permitted repair actions (Remark 2). Its role also differs from that of global rescheduling: it answers whether feasibility can be restored *quickly* while confining rewrites to the set, not whether a better makespan can be found. Confining rewrites matters because every rewritten occupancy is a route or time window that must be reissued to a vehicle already executing the plan; keeping the rest unchanged is a form of schedule stability [24].

Contributions.

C1. A class of invalidation the operation-precedence graph cannot capture. Under corridor blockage and slowdown, operation-level scoping returns an empty scope although the original schedule is infeasible (50/50, Section 6.2); the root cause is that the operation-precedence graph has no yielding relations among occupancies (Proposition 1).

C2. The reservation impact set as the rescheduling scope. STRC delineates the scope by closure over yielding and same-vehicle, same-job, and same-machine succession. Membership is uniquely determined and no dependency edge leaves the set. The set is nonempty under all four disturbance types and, under Class A disturbances, contains every occupancy already included by the operation-precedence graph (Section 6.6). With the same rerouting, it rewrites fewer occupancies than rewriting all unfinished ones (fewer/equal/more on 31/18/1 pairs; Section 6.7).

C3. A matching local recovery with the sources of each result separated. Under the same network, router, and checker, scoping on the operation-precedence graph yields an executable schedule on 0/50 corridor-blockage pairs, scoping on the impact set on 50/50 (Section 6.2). Bounded repair takes milliseconds, but its makespan is no better than that of global rescheduling under every tested time limit, and relaxing the limit does not reverse this order (Section 6.5); against rewriting all unfinished occupancies, runtime and makespan are almost identical, so the millisecond runtime comes from single-pass rerouting, not from the impact set (Section 6.7).

Integrated scheduling before execution uses the reservation table to build a conflict-free plan; this paper uses the same table at execution time to locate the invalidated scope.

Section 2 reviews related work; Section 3 states the setting and the recovery problem; Section 4 defines STRC and its properties; Section 5 describes bounded repair; Section 6 reports the experiments; Section 7 concludes.

2 Related work

A basic question in dynamic scheduling is how to delineate the rescheduling scope: which operations, corridor occupancies, or vehicles must be replanned after a disturbance, while everything outside the scope keeps the original plan. We review four classes of objects on which the scope has been delineated and compare them with this paper in Table 1.

2.1 Local rescheduling in job shops

For operation-level events, delineating the scope by the affected operations is still common in dynamic scheduling with AGVs. Zhang et al. [26] reschedule the operations not yet processed after a machine breakdown; Tang et al. [22] classify disrupted operations and delivery tasks as retained, continued, or reconstructed, and compare full rescheduling, partial rescheduling, and right shift. AOR and MAOR [18] reschedule only the operations directly or indirectly affected. Delineating the scope along the time axis and enlarging it after failure have also been studied, including match-up, which reschedules only up to a point where the original schedule can be resumed [15, 17], and Local Rescheduling with incrementally enlarged time windows [6]; right shift instead postpones the entire unfinished part. Rolling-horizon

methods solve overlapping shorter-horizon subproblems in sequence [8], and decomposition methods split the job set along its precedence structure [11]; see [14, 24] for surveys. Delineating a scope and rescheduling locally within it is thus a fairly general practice, but these works start the delineation from disturbed operations; when a disturbance only makes a corridor impassable, there is no such operation, so the scope is necessarily empty although the schedule is infeasible.

2.2 Occupancy order on exclusive resources

The alternative graph of Mascis and Pacciarelli [12] generalizes the disjunctive graph: vertices are occupations of block sections, fixed arcs encode route precedence, and paired alternative arcs encode which train passes first; D'Ariano et al. [1] use it for real-time train rescheduling. An alternative arc captures the order in which two trains pass the same block section, the same kind of occupancy order as our yielding relation. In the alternative graph this order is a decision variable, chosen so that no positive cycle arises and the objective is optimized; railways treat block sections as machines, so an unavailable section appears directly on the graph and the problem studied here does not arise. We instead use the already fixed order only to determine which occupancies must be replanned after the disturbance.

2.3 Local replanning in conflict-free multi-vehicle routing

In multi-agent path finding (MAPF), replanning only the affected agents is standard. Švancara et al. [21] compare Replan All, Replan Single, and Replan Affected, determining the affected agents by independence detection; Malka et al. [10] replan only the agents affected by observed map changes; lifelong MAPF treats spatiotemporal occupancy as a first-class constraint [27]. MAPF, however, has no job–operation–machine assignment, so operation-level scoping cannot miss a blockage there, and its affected set is the output of a detection algorithm that varies with the implementation, whereas the reservation impact set is fixed once the edges are given (Section 4.2).

2.4 Flexible job shop scheduling with transportation

Much recent work still takes travel times as a constant matrix [13, 16, 25] or as load-dependent times with an aggregate congestion delay [4]; neither records corridor occupancies, so a corridor blockage has no counterpart. Congestion-aware dispatching selects vehicles by proxies such as local density [5]. Van Os and Basten [23] formalize flexible job shop scheduling with conflict-free transportation and solve most benchmark instances to optimality; two-stage schemes first fix production and then resolve vehicle conflicts [20]; a learning-based scheduling layer can be coupled with a conflict-resolving routing layer [7]. These works build a conflict-free schedule *before* execution and do not handle execution-time disturbances (e.g., Sun et al. [20] assume that AGVs are always available), whereas the dynamic scheduling works with AGVs in Section 2.1 do not model corridor contention (e.g., Zhang et al. [26] ignore AGV collisions). In flexible job shop scheduling with transportation, a reservation table and execution-time disturbances have thus not yet appeared together; when both are present, the only directly applicable practice is operation-level scoping, which yields an empty scope for a corridor blockage (50/50, Section 6.2).

As Table 1 shows, none of the listed works combines operation and machine assignment, occupancy time windows, and execution-time disturbances. Our scope puts "no misses" first: every invalidated occupancy must be included, while occupancies unreachable along the propagation channels keep their original plan; the set is not meant to be minimal, since a minimal set depends on the repair actions (Remark 2). The channels must match those along which invalidation actually propagates: AOR follows only jobs and machines and thus obtains an empty scope under a blockage, while relations that do not propagate invalidation would only enlarge the scope. Our four channels—corridor (yielding),

Table 1. Comparison of related work and this paper in problem setting and rescheduling scope. ✓: present; ✗: absent; —: not applicable (no local rescheduling scope is delineated). "Occupancy time windows" means that the model explicitly records the intervals during which each vehicle (train) occupies a segment (block section, spatiotemporal cell). "Scope on" is the object that may be rewritten inside the scope; "Propagation channels" are the channels tracked during delineation, named by the shared resource through which invalidation passes; "No misses" means that, under a disturbance acting only on corridors (e.g., a blockage or slowdown), all invalidated occupancies fall inside the scope; "Repair-independent" means that the scope is uniquely determined by a given rule and does not vary with the implementation or stopping condition of the repair or detection algorithm.

	Problem setting			Rescheduling scope			
Representative work	**Operation & machine assignment**	**Occupancy time windows**	**Execution-time disturbances**	**Scope on**	**Propagation channels**	**No misses**	**Repair-independent**
AOR [18]	✓	✗	Operation events	Affected operations	Job, machine	✗	✓
Match-up [15, 17], LRS [6]	✓	✗	Operation events	Time window	Window enlarged stepwise	✗	✗
Dynamic scheduling with AGVs [22, 26]	✓	✗	Equipment failures, rush orders	Operations and transport tasks	By disturbance category	✗	✓
Real-time railway rescheduling [1, 12]	✗	✓	Train delays	All trains	— (global rescheduling)	✓	—
Online MAPF [10, 21]	✗	✓	New tasks, map changes	Affected vehicles	Inter-vehicle path conflicts	✓	✗
Constant or aggregate transport time [4, 13, 16]	✓	✗	None (before execution)	—	—	—	—
Integrated conflict-free scheduling [7, 20, 23]	✓	✓	None (before execution)	—	—	—	—
This paper	✓	✓	**Operation events and corridor disturbances**	**Corridor occupancies (assignment and sequence unchanged)**	**Corridor, vehicle, job, machine**	✓	✓

vehicle, job, and machine—correspond to the shared resources that constrain occupancy timing; inside the scope, routes and intervals (and, under a vehicle failure, vehicles) may change, while machine assignment and processing sequence stay fixed.

3 The corridor-reservation setting and the execution-time recovery problem

3.1 System setting and notation

The problem belongs to flexible job shop scheduling with conflict-free transportation constraints, so we build on the formulation of van Os and Basten [23]; the two differ in what is reserved (Section 3.1.2). The pre-disturbance schedule $\mathcal{S}^0$ is a given input; delineation depends only on $\mathcal{S}^0$ itself (assignment, sequence, and reservation table), not on how it was obtained, so $\mathcal{S}^0$ may come from any scheduling method.

3.1.1 Shop floor and four groups of decisions. The shop routing graph $G = (V, E)$ is strongly connected. Each machine m processes through a robotic arm at node $\mathrm{loc}(m)$, so a robotic-arm failure means a failure of that machine; $v_0 \in V$ is the load/unload station. There are n jobs, a machine set M, and a homogeneous fleet of unit-load vehicles $K = \{1, \dots, N_A\}$. Job j has a fixed chain $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} has candidate machines $\Omega_{ji} \subseteq M$ and processing time t^P_{jim} on machine m. A schedule fixes (i) the machine assignment x; (ii) the processing sequence π on each machine; (iii) the vehicle assignment y of each transport; and (iv) a conflict-free path for each vehicle move. We write $\mathcal{S} = (x, \pi, y, R)$, where the reservation table R records group (iv). Transports are not fixed parameters but are determined by the machine assignment: a transport is needed only when two consecutive operations of a job are assigned to machines at different

Table 2. Frequently used notation

Symbol	Meaning
$G = (V, E)$, τ_e	Corridor network; traversal time of corridor e
x, π, y	Machine assignment, processing sequence, vehicle assignment
R, $r = (e, [t^s, t^e), k, u)$	Reservation table; a half-open occupancy (vehicle k, leg label u)
G_T / G_R	Operation-precedence graph / reservation dependency graph
$\mathcal{S}^0$, $\mathcal{S}'$	Schedule before the disturbance, schedule after recovery
$\mathcal{D}$, t_{now}	Disturbance event; decision time
T_{direct}, T_{impact}	Directly disturbed operations / θ-step scope on G_T
$\text{Src}(\mathcal{D})$, U, R°	Seed occupancies; rewritable set; unfinished occupancies
$\text{Cl}(\cdot)$	Reservation impact set (obtained by STRC)
φ	Fraction of disturbed future operations (magnitude sweep)

nodes. Each transport is split into an *empty leg*, in which the vehicle travels unloaded from its current position to the machine of the preceding operation to pick up the job, and a *loaded leg*, in which it carries the job to the machine of the succeeding operation; each leg has its own task label u. Loading and unloading times are zero. Rewritability, keeping, and rerouting are all decided per transport leg, not per transport or operation.

3.1.2 Corridor reservation and conflict freedom. Each corridor $e \in E$ is an exclusive resource with traversal time τ_e, treated as a single lane: a vehicle in either direction occupies the whole corridor, so neither head-on encounters nor overtaking can occur, as with the narrow aisles and zone control common in AGV shops (a two-lane corridor can be modeled as two corridors). Occupancies are half-open intervals, so a following vehicle may enter exactly when the preceding one leaves; vehicles wait only at nodes, which have ample parking capacity, and never inside a corridor. A reservation record $r = (e,\ [t^s, t^e),\ k,\ u) \in R$ means that vehicle k occupies e during $[t^s, t^e)$ for leg u, and distinct occupancies of the same corridor must be disjoint. Paths use only time slots not yet occupied in R, so the table stays conflict-free without a posterior check. Time-window routing is the basic module shared by our method and all baselines: given origin, destination, earliest departure, and the current reservation table R, it returns a path whose occupancies overlap no existing one. We implement it as time-window Dijkstra [20] and change only *which* legs are handed to it. Van Os et al. reserve nodes [23] and Lyu et al. add node capacities [9]; we reserve only corridors (network edges), so our results are not directly comparable with their reported values (Section 7.1).

3.1.3 Operation-precedence graph and reservation dependency graph. The operation-precedence graph $G_T = (T_T, E_T)$ has operations as vertices and two kinds of directed edges: job precedence, from O_{ji} to $O_{j,i+1}$, and same-machine precedence, from an operation to the one processed immediately after it on the same machine; it contains no occupancies and hence no yielding relations. The reservation dependency graph $G_R = (R, E_R)$ has corridor occupancies as vertices; an edge $r \to r'$ means that invalidating r may force r' to be rerouted. Its yield and same-vehicle edges link consecutive occupancies of the same corridor and of the same vehicle, and its same-job and same-machine edges carry the precedences of G_T over to the *transport occupancies* of operations, i.e., the occupancies of the legs that move the job for that operation (Definition 3). The vertex sets are disjoint; hence a disturbance that changes only when corridors are passable yields no seed on G_T, whereas on G_R every planned occupancy overlapping the disturbed interval—exactly what makes the schedule infeasible—is a seed (Figure 2; notation in Table 2).

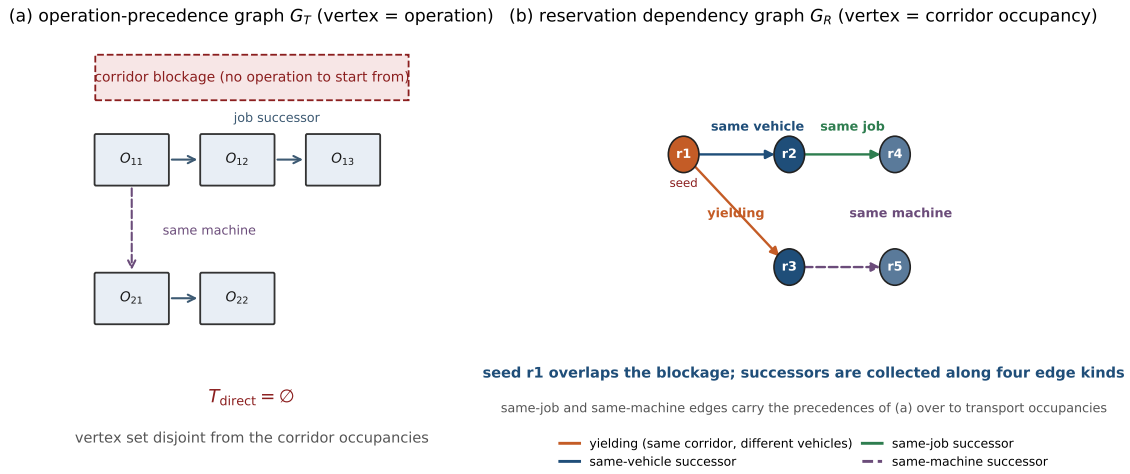


Fig. 2. The two graphs have disjoint vertex sets. (a) Operation-precedence graph G_T: a corridor blockage finds no seed operation. (b) Reservation dependency graph G_R: starting from the seed occupancy overlapping the blockage, subsequent occupancies are included along yielding, same-vehicle, same-job, and same-machine edges; the same-job and same-machine edges carry the precedences of (a) over to transport occupancies.

3.2 Execution-time assumptions

A1–A4 are shared by all compared methods, so that differences can be attributed to the scope rather than the model; A5 constrains only our method, while the global-rescheduling baseline may also change the machine assignment and processing sequence.

A1. Feasible initial schedule. $\mathcal{S}^0$ passes the conflict-freedom check before the disturbance.

A2. No backtracking on finished occupancies. Occupancies with $t^e \le t_{\text{now}}$ are not rewritable.

A3. Single, fully observed disturbance. Successive disturbances, estimation errors in disturbance information, and repeated rescheduling as new information arrives are out of scope.

A4. Exclusive corridors, ample buffers, homogeneous unit-load fleet. A blockage is injected into the routing layer as an interval during which the corridor may not be entered (the *blockage window*); finite buffers, charging, acceleration, and loading durations are not modeled.

A5. Permitted recovery actions. For Class B, x and π are kept and only rerouting is performed, with scope escalation upon failure. Class A additionally allows reassigning a failed vehicle's tasks to another vehicle, or a failed machine's unfinished operations to other feasible machines. π is never changed and the search is not restarted; the global-rescheduling baseline may change x and π (Section 5.3).

3.3 A dichotomy of disturbances

Definition 1 (Class A/B disturbances). *A disturbance is of **Class A** if it directly invalidates the assignment or* executability *of some operations (robotic-arm failure, processing-time deviation, rush-order insertion, vehicle failure). It is of **Class B** if it only makes some corridors unavailable for some period without making any operation inexecutable (corridor blockage, corridor slowdown).*

Remark 1 (Why vehicle failure is Class A). *A vehicle failure changes no machine assignment, but the operations the vehicle transports cannot start as planned. Through the mapping "vehicle→operations it transports", G_T still contains seed operations. This holds only if the method records which operations each vehicle transports; otherwise vehicle failure also yields an empty scope. The baseline in Section 6.6 records this correspondence.*

Definition 2 (Scopes on the operation-precedence graph and seed occupancies). *$T_{\text{direct}}(\mathcal{D})$ is the set of operations that cannot be executed as planned; for Class B, $T_{\text{direct}} = \varnothing$ by convention. $T_{\text{impact}}(\mathcal{D}, \theta)$ collects the operations reached from T_{direct} on G_T within at most θ steps along job and same-machine successor edges (including T_{direct}). The seed set $\text{Src}(\mathcal{D}) \subseteq R$ contains only occupancies not yet finished at t_{now} ($r.t^e > t_{\text{now}}$) and is given by disturbance type: (a) **corridor blockage/slowdown** $\mathcal{D} = (e, [t_a, t_b))$: $\{ r : r.e = e,\ [r.t^s, r.t^e) \cap [t_a, t_b) \neq \varnothing \}$ (for multiple windows, the union); blockage and slowdown share this rule and differ only in rerouting: a blocked corridor is impassable, while a slowed one stays passable with the traversal time inside the interval lengthened; (b) **vehicle failure** $\mathcal{D} = (k)$: $\{ r : r.k = k \}$; (c) **robotic-arm failure** $\mathcal{D} = (m, F)$: for each job, all unfinished occupancies from its smallest operation index in F onward, because the inbound transport may have finished, and taking only the operation itself would make the impact set cover fewer operations than the affected set on G_T.*

Proposition 1 (The operation-precedence graph yields an empty scope for Class B). *Let $\mathcal{D}$ be a corridor blockage. Then $T_{\text{impact}}(\mathcal{D}, \theta) = \varnothing$ for every finite $\theta \geq 0$. If, in addition, $\text{Src}(\mathcal{D}) \neq \varnothing$, then $\mathcal{S}^0$ is infeasible under $\mathcal{D}$.*

Proof. Since the blockage window is finite and buffers are ample (A4), vehicles can wait at nodes until the blockage ends, so no operation becomes inexecutable and $\mathcal{D}$ is of Class B; hence $T_{\text{direct}} = \varnothing$ and the θ-step expansion is empty. If $r \in \text{Src}(\mathcal{D})$ exists, r overlaps the blockage window on the same exclusive corridor, which violates conflict freedom (A4); hence $\mathcal{S}^0$ is infeasible. □

The first part follows from the convention; the nontrivial part is the second. Hence "an empty scope on $G_T \Rightarrow$ no rescheduling needed" fails when corridors are reserved by time windows, independently of any repair algorithm; Section 6.2 verifies this on 50/50 pairs.

3.4 The recovery problem

Problem 1 (Bounded rescheduling with the scope delineated on the reservation table). *Given $\mathcal{S}^0$, $\mathcal{D}$, and t_{now}, find $\mathcal{S}'$ such that (i) $\mathcal{S}'$ is conflict-free under $\mathcal{D}$; (ii) occupancies outside the rewrite scope keep their corridors, intervals, and vehicles as in $\mathcal{S}^0$ as far as possible; (iii) the runtime is subject to a budget. The secondary objective is the makespan $C_{\max}(\mathcal{S}')$.*

Bounded rescheduling first delineates the rewritable set $U \subseteq R$ and then reroutes only the transports in U; we call the second step *bounded repair* (Section 5). We take $U = \text{Cl}(\text{Src})$; the baseline instead takes the transport occupancies of the operations in T_{impact} that are unfinished at t_{now}, which is empty under Class B by Proposition 1. The two are denoted R2 and R1 in the experiments.

4 The spatiotemporal reservation closure

This section determines U: we build a dependency graph over occupancies and take all occupancies reachable from the seeds $\text{Src}(\mathcal{D})$ along its edges.

4.1 Reservation dependency edges

Definition 3 (Reservation dependency graph). *The vertices are the occupancies in R; an edge $r \to r'$ means that once r is rewritten, r' may have to be rewritten as well. There are four kinds of edges: (1)* ***yield edges*** *between occupancies of the same corridor by different vehicles whose intervals overlap* or abut *($r.t^e = r'.t^s$), from the earlier entry time t^s to the later one—meeting end to start is legal, but if the former is delayed the latter must give way; (2)* ***same-vehicle successors****, adjacent in time, from the earlier to the later; (3)* ***same-job successors****, from the transport occupancies of operation i to those of operation $i+1$; (4)* ***same-machine successors****, from the transport occupancies of an operation to those of the operation processed next on the same machine. This yields $G_R = (R, E_R)$. The edge set is a modeling choice that may over-include or miss dependencies (Remark 2).*

Definition 4 (Reservation impact set and STRC). *Given seeds $S \subseteq R$ and t_{now}, let $R^\circ = \{\, r \in R : r.t^e > t_{\text{now}} \,\}$ be the unfinished occupancies and G_R° the subgraph that keeps only them and the edges among them. The reservation impact set is*

$$\mathrm{Cl}(S) = \left\{ v \in R^\circ : \exists\, s \in S \cap R^\circ \text{ such that } s \to^* v \text{ in } G_R^\circ \right\},$$

i.e., all unfinished occupancies reachable from the unfinished seeds, seeds included; computing it is called the spatiotemporal reservation closure (STRC). With adjacency lists, one breadth-first search computes $\mathrm{Cl}(S)$ in $O(|R^\circ| + |E_R^\circ|)$ time.

4.2 Closure

Proposition 2 (Structural closure). *For any $S \subseteq R$, G_R° contains no edge $u \to v$ with $u \in \mathrm{Cl}(S)$ and $v \in R^\circ \setminus \mathrm{Cl}(S)$.*

Proof. If such an edge existed, then $s \to^* u \to v$ for some unfinished seed s, so $v \in \mathrm{Cl}(S)$, a contradiction. □

Closure is thus a corollary of the definition once the edge set is fixed; its physical completeness depends on the choice of edges.

Remark 2 (The reservation impact set and the minimal rewritable set). *Closure neither guarantees that the impact set suffices to restore feasibility nor that it is small; we relate it to the minimal rewritable set in three points. First, a* minimal *rewritable set is defined only relative to a repair capability (rerouting only, or also reassignment), whereas* Cl *is determined by G_R alone and is not meant to coincide with any of them. Second, the edges over-include: a yield edge is created for any overlap without checking whether the later vehicle could detour, and same-vehicle edges may pull in chain tails the delay never reaches. In the opposite direction, the known omission—abutting intervals—has been folded into the yield edges; a difference probe (on the 50 pairs of E3, each impact-set occupancy is delayed alone by one time unit, and a leak is counted whenever it newly overlaps an occupancy outside the set on the same corridor) finds 0 leaks (0/50 pairs; in 44 of them the set leaves occupancies outside), which tests this omission but does not prove completeness. Third, empirically the impact set covers about 92% of the unfinished occupancies (Section 6.6), less than one tenth below rewriting all of them. Bounded repair is fast mainly because it does not restart the search, not because far fewer occupancies are to be rewritten; "bounded" does not mean "small". This small difference still lowers the rewrite fraction by 0.064 on average (Section 6.7). The impact-set share does not rise with instance size (Section 6.9), and static network indicators fail to predict it (Section 6.4).*

5 Bounded repair on the reservation impact set

Figure 3 outlines the procedure: delineate U by the reservation impact set, replan only the occupancies inside it while keeping the rest as planned, and escalate U if the result is still infeasible. All methods share the same corridor network, time-window routing, and conflict-free check.

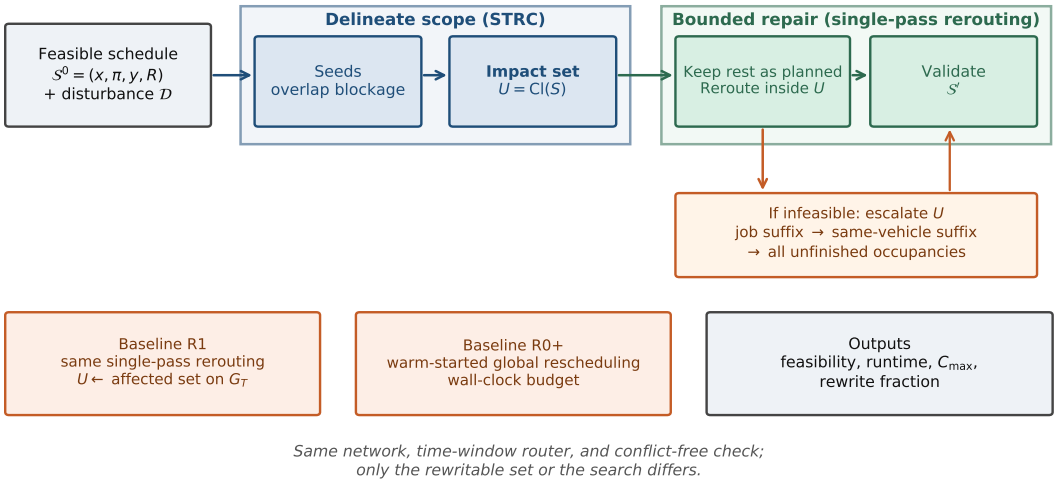


Fig. 3. Bounded repair scoped on the reservation impact set. Top row: the impact set is obtained from the seeds and vehicles are rerouted; middle row: the rewritable scope is escalated after failure; bottom row: the baseline scoping on the operation-precedence graph, the warm-started global rescheduling baseline, and the output metrics. Three further millisecond baselines (RS, RD, RA) are not drawn (Section 6.7).

Algorithm 1 Single-pass rerouting ReplayReroute

Require: schedule $\mathcal{S}^0 = (x, \pi, y, R)$, disturbance $\mathcal{D}$, rewritable set U, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure

1: write $\mathcal{D}$ into the routing layer ▷ blockage: blockage window; slowdown: traversal time inside the window lengthened
2: $R' \leftarrow \varnothing$
3: **for** each occupancy not in U (including all finished occupancies) **do**
4: pre-commit its original fields $(e, [t^s, t^e), k)$ to R'; **if** it conflicts with a blockage window **then return** failure
5: **end for**
6: **for** each $r \in U$, in the original processing order **do**
7: route from the vehicle's node at t_{now} on the current R' and commit the new occupancies
8: update the machine idle time and job ready time concerned according to π
9: **end for**
10: **if** independent check passes (no conflicts, fields outside U unchanged) **then return** $\mathcal{S}' = (x, \pi, y, R')$ **else return** failure

5.1 Single-pass rerouting

Given U, single-pass rerouting keeps the vehicle and machine assignments and reschedules the transports in U one by one in the original processing order (Algorithm 1). It works per occupancy: the part of a trip traversed before t_{now} is finished and pre-committed unchanged, and planning starts from the vehicle's node at that time. By complexity, delineation needs one breadth-first search, whereas the replay calls the time-window router once per leg in U, which dominates the runtime. No population-based search is launched. Section 6.2 fixes this procedure and replaces only the source of U, so that differences in feasibility can be attributed to the scope object.

Algorithm 2 Bounded repair on the reservation impact set (single-pass rerouting + scope escalation)

Require: schedule $\mathcal{S}^0$, disturbance $\mathcal{D}$, time t_{now}
Ensure: schedule $\mathcal{S}'$ or failure
1: $U \leftarrow \text{Cl}(\text{Src}(\mathcal{D}))$
2: **for** round = 0, 1, 2, 3 **do**
3: $\quad \mathcal{S}' \leftarrow \textsc{ReplayReroute}(\mathcal{S}^0, \mathcal{D}, U, t_{\text{now}})$; **if** $\mathcal{S}'$ is feasible **then return** $\mathcal{S}'$
4: $\quad$ update U by the scope-escalation rule ▷ job suffix / same vehicle / all unfinished
5: **end for**
6: **return** failure

Proposition 3 (A sufficient condition for unchanged fields outside the set). *Let $U = \text{Cl}(S)$, and let single-pass rerouting (Algorithm 1) satisfy: (i) the blockage windows are written into the routing layer; (ii) for every $r \in R^\circ \setminus U$, its fields $(e, [t^s, t^e), k)$ are pre-committed to the new table and the pre-commit succeeds; (iii) the time-window router replans only the occupancies in U; (iv) the final schedule passes the conflict-free check. Then the corridor, interval, and vehicle of every $r \in R^\circ \setminus U$ in the recovered table agree with $\mathcal{S}^0$. The proposition holds per occupancy, not per whole transport.*

Proof. By (ii), occupancies outside the set enter the new table with their original fields before the replay; by (iii), the replay neither deletes nor rewrites them; the check in (iv) only rules out conflicts and alters no committed field. □

Premises (i)–(iv) are execution conditions that can be checked against Algorithm 1; beyond them, the proposition relies on A1, A2, and A4. A failed pre-commit indicates an incomplete seed set. If scope escalation enlarges U, the proposition holds only for the occupancies finally left outside. Premise (iii) fails if rerouting is decided at a unit coarser than an occupancy, so the implementation must decide it per occupancy, and for a leg spanning t_{now} must keep the segments already traversed (otherwise finished occupancies are rewritten, which a check of the fields outside U cannot detect).

5.2 Scope escalation after failure

Escalating the scope has precedents: Local Rescheduling [6] relaxes the time window step by step, and AOR adds successor operations (Section 2.1); we escalate along reservation dependencies, called *scope escalation*. E3 disables this step (Section 6.2). If, with all occupancies outside U fixed, those in U find no feasible path and interval (e.g., a transport cannot arrive when its operation requires), the replay fails; we then enlarge U and retry: (i) add the unfinished occupancies in the same-job suffixes of operations in U; (ii) add those in the same-vehicle suffixes of the vehicles involved; (iii) set $U \leftarrow R^\circ$, the rewritable set of RA. Each round remains a deterministic single pass (Algorithm 2).

5.3 Baselines

The global rescheduling baseline R0+ is a genetic algorithm that encodes (x, π) as a chromosome, is warm-started with $\mathcal{S}^0$ as one individual of the initial population, and searches under the same blockage windows within a wall-clock budget. One *decoding* routes all transports from scratch on the table with the blockage windows written in; finished occupancies are pre-committed, and only operations not yet started are scheduled from t_{now}. Because R0+ can reassign and resequence, it is more likely to shorten $C_{\max}$, but its runtime is set by the budget. Bounded repair is therefore not expected to win on makespan; its strength is runtime. To test whether scoping on the impact set matters *within* a millisecond runtime, Section 6.7 adds three millisecond baselines (RS, RD, RA), of which RA differs from our method only in which occupancies are rewritable.

6 Experiments

6.1 Experimental setup

Instances. The main batch has five instances (jobs × machines × AGVs): a small hand-crafted $3 \times 3 \times 2$ instance, a congested hand-crafted $8 \times 4 \times 4$ instance, and three generated $8 \times 4 \times 4$ spine layouts, funnel, high, and mid. In a spine layout, a main corridor runs through the machine area, machines hang on two hubs via branches, and the load/unload station sits at one end with one or two parallel exits [19], so all transports share the same exit and middle section; the congested instance also has this layout. The three spine layouts differ only in the number of parallel lanes (funnel: one exit and one middle lane; high: two exits; mid: two exits and two middle lanes), i.e., in capacity, not in distance. We use ten run seeds fixed before the experiments (42, 7, 2024, 99, 123, 13, 1, 777, 31415, 8; E5 uses the first three), giving $5 \times 10 = 50$ instance–seed pairs. All per-pair comparisons and paired tests use the instance–seed pair as the unit: in the same pair, all methods read the same initial schedule, blockage, and decision time. The initial schedule is obtained by replaying the time-window router in processing order and passes the independent check.

Second batch with published layouts. To test whether the conclusions depend on the layouts we constructed, a second batch of 5 instances (50 pairs) transcribes the grid size, the nodes of the station and machines, and the adjacent node pairs without a corridor from Appendix A of Lyu et al. [9], checked by script against the encoding shipped with the van Os toolkit. All other parameters are matched to the main batch, so the two batches differ only in the layouts. Its six layouts (layouts 1–6, with 3 to 8 machines) are used except layout 1, which cannot give every operation two candidate machines at $f = 0.6$. The five remaining layouts lie on only 2 distinct networks (layouts 2–3: a 4×4 grid of 16 nodes with 23 of its 24 corridors present; layouts 4–6: a 5×5 grid with 38 of 40), so along network shape there are 2 independent samples. Lyu et al. give per-edge durations only for one example, so we use equal weights; following the single load/unload station common in joint machine–AGV scheduling [2], the loading and unloading stations are merged into one, which leaves the network unchanged; and their job data are not used, because in public instance families transport takes only a few percent of processing time and corridor contention would be unobservable. We therefore borrow only their networks and do not compare with the values they report.

Disturbances. All experiments use $t_{\text{now}} = 0.35\, C_{\max}(\mathcal{S}^0)$, fixed in advance: about 40% of the occupancies have then finished and 60% remain (no sensitivity analysis on this value). E1–E3, E5, and E7 block the corridor with the most unfinished occupancies at t_{now}, in one continuous window covering all of them (a *busy-corridor blockage*); E6 adds three further types; E4 blocks a corridor in the load/unload minimum cut. The magnitude sweep sorts operations with unfinished occupancies by their earliest such occupancy, takes the first fraction φ, and blocks each of their unfinished occupancies on its own corridor and interval, without merging blockages, so that no unselected occupancy is covered.

Metrics. An occupancy counts as *rewritten* if its corridor, interval, or vehicle after repair differs from $\mathcal{S}^0$; the *rewrite fraction* is the share of rewritten occupancies in the whole table R. It differs from the *rewritable share* $|U|/|R^\circ|$, which counts the occupancies a method is allowed to change. Makespan *degradation* is $(C_{\max}(\mathcal{S}') - C_{\max}(\mathcal{S}^0))/C_{\max}(\mathcal{S}^0)$. A *finished-occupancy violation* is a rewritten occupancy with $t^e \le t_{\text{now}}$, which A2 forbids.

Methods and conventions. Table 3 lists the methods in increasing order of their difference from R2. In R1, θ is the maximum number of forward hops along the job precedence; it matters only for Class A, since a Class B disturbance hits no operation and the scope is empty for any θ (Proposition 1). R1 transplants AOR scoping [18] into our reservation setting: the θ-hop truncation and the mapping from operations to reservations are ours, since the original setting has no reservation table; its results reflect the rule in this setting, not the original method in its own. R1/R2 differ only in the scope *object*, RA/R2 only in *whether* the scope is restricted; RD decodes the original chromosome once, whereas R0+

Table 3. Compared methods. All share the network, time-window router, initial schedule, and independent checker, and none rewrites finished occupancies.

Method	Rewritable scope	Recomputation	Termination
R2 (STRC)	Reservation impact set Cl(Src)	Single-pass rerouting	Single pass
R1	Affected set on G_T ($\theta = 2$) mapped to occupancies	Single-pass rerouting	Single pass
RA	All unfinished occupancies R°	Single-pass rerouting	Single pass
RS	None delineated; all unfinished occupancies shifted	Uniform right shift	Single pass
RD	None delineated	One fixed-prefix decoding	Single pass
R0+	None delineated	Warm-started GA (population 40)	Wall-clock budget

searches repeatedly within its budget. Only R0+ terminates on a budget; all others are deterministic single passes and can be compared with one another directly. Scope escalation is disabled in E1–E3, E5, and E7, so that a fallback to all unfinished occupancies cannot mask the difference between scope objects; its end point is RA, measured separately. It is enabled only in the magnitude sweep and in E8. The main batch establishes the phenomena under full control of the instance parameters; the second batch only tests whether they depend on our layouts, and is pooled with the main batch only in E5.

Implementation and statistics. All code is pure Python (standard library only), run single-threaded on an Intel Xeon W-1270 with 64 GB RAM under Windows 10; runtime conclusions are stated as ratios or orders of magnitude. The paper contains two inferential tests, both in Section 6.7: R2 versus RA on rewrite fraction (primary) and R2 versus RD on makespan, each a two-sided Wilcoxon signed-rank test paired by instance–seed pair; Holm correction does not change either conclusion. The primary test is accompanied by a Hodges–Lehmann estimate with a bootstrap 95% CI (10^4 resamples). All other counts, means, and medians are descriptive.

6.2 E1–E3: empty scope, containment, and boundary ablation

Table 4 summarizes the per-instance statistics. Over the 50 pairs: (E1) the three conditions of Proposition 1—$T_{\text{impact}} = \varnothing$, a nonempty seed set, and an infeasible original schedule—hold together in 50/50 pairs; the first follows from the Class B convention, and the substantive result is that the original schedule is infeasible every time, so "empty scope, hence no rescheduling" never holds. (E2a) The impact set is closed under outgoing edges in 50/50 pairs, confirming that the implementation matches Proposition 2. (E2b) After single-pass rerouting, the corridor, interval, and vehicle of every occupancy outside the set are unchanged in 50/50 pairs, i.e., the premises of Proposition 3 are met in our experiments; a separate check of all finished occupancies finds none rewritten. (E3) With single-pass rerouting fixed and escalation disabled, R1 is feasible in 0/50 pairs and R2 in 50/50. The two arms share the rerouting procedure, initial schedule, decision time, and blockage; **the only difference is which occupancies are rewritable**. R1's rewritable set is empty, so the replay leaves the schedule unchanged and it still occupies the blocked corridor: R1 fails because it decides that no rescheduling is needed, not because of repair quality. The median runtime of single-pass rerouting is 3.2 ms (maximum 12.8 ms).

|Cl| grows with the instance, while Cl/|R| stays within 0.56–0.59 (pooled median 0.571, per-instance IQR 0.024–0.049) and does not distinguish the three spine layouts; between-instance variation is of the same order as between-seed variation. We rerun E1 and E3 under three non-uniform edge weightings (independent lognormal factors with geometric mean 1; corridors slower with distance from the station, up to 1.8×; each corridor randomly halved or doubled): E1 is

Table 4. E1–E3 per instance (means over 10 seeds). $|R|$: occupancies in the whole table; $\mathrm{Cl}/|R|$: share of the reservation impact set. On every instance, all checks of E1, E2a, E2b and R2 feasibility pass in 10/10 pairs and R1 is feasible in 0/10; totals are given in the text. funnel: one exit and one middle lane; high: two exits; mid: two exits and two middle lanes.

Instance	Mean $\|R\|$	Mean $\|\mathrm{Src}\|$	Mean $\|\mathrm{Cl}\|$	$\mathrm{Cl}/\|R\|$	R2 ms
Small	28.6	5.6	16.9	0.59	0.66
Congested	85.0	15.0	48.4	0.57	2.84
funnel	149.1	15.5	85.4	0.57	5.09
high	150.5	15.5	88.3	0.59	3.40
mid	147.6	15.0	82.8	0.56	3.96

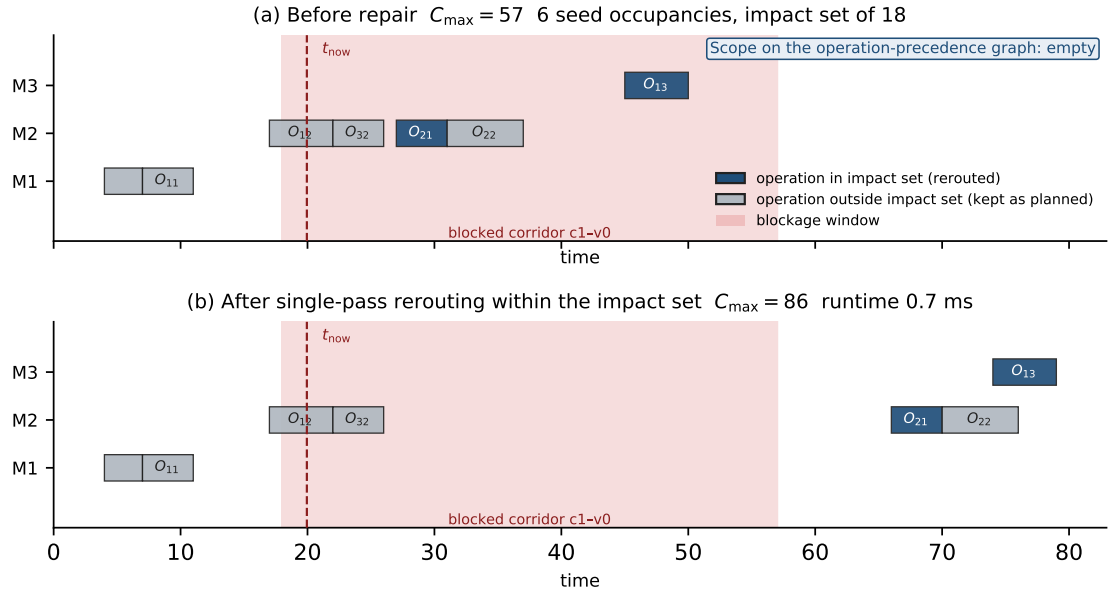


Fig. 4. Case Gantt chart ($3 \times 3 \times 2$ small instance, run seed 42, busy-corridor blockage). (a) Original schedule; blue operations have transports in Cl, gray ones are kept; the dashed line marks t_{now} and the band the blockage window. (b) After single-pass rerouting.

unchanged and R1 remains infeasible everywhere, while R2 is feasible in all pairs under the first two and in 49/50 under the third, so neither result depends on equal edge weights.

6.3 Case Gantt chart

Figure 4 shows one example: the $3 \times 3 \times 2$ small instance (chosen so that every operation is visible), run seed 42, a busy-corridor blockage, and otherwise the E3 setup: the scope on G_T is empty ($T_{\mathrm{impact}} = \varnothing$), whereas there are 6 seed occupancies and 18 occupancies in the reservation impact set. After single-pass rerouting, the operations whose transports lie in the impact set are rearranged, the others stay in place, and the schedule regains feasibility in under a millisecond ($C_{\max} : 57 \to 86$). The figure illustrates the form of the local change and is not used to compare makespans.

Table 5. E4: static network indicators and impact-set share (one LU-cut corridor blocked, ten seeds per layout; median Cl/|R|). All instances share every generation parameter except network and machines. "Range": maximum minus minimum share within a group (groups of 2, 3, and 3 layouts, so only a rough comparison).

Source and grid	Layout	Machines	Nodes/corridors	LU cut	Funnel	Cl/|R|	Range
Lyu et al., 4 × 4	Layout 2	4	16/23	2	0	0.512	
Lyu et al., 4 × 4	Layout 3	5	16/23	2	0	0.305	0.207
Lyu et al., 5 × 5	Layout 4	6	25/38	2	0	0.385	
Lyu et al., 5 × 5	Layout 5	7	25/38	2	0	0.422	
Lyu et al., 5 × 5	Layout 6	8	25/38	2	0	0.460	0.075
Ours	funnel	4	9/8	1	0.286	0.491	
Ours	high	4	10/10	2	0.286	0.497	
Ours	mid	4	11/12	2	0.286	0.446	0.051

6.4 E4: can network-structure indicators predict the impact-set size? (exploratory)

We test whether three indicators computed from the network alone predict the impact-set share, blocking one corridor in the LU cut: the *LU cut* (minimum number of corridors disconnecting the load/unload station from all machines), the *funnel share* (share of the mean station-to-machine shortest-path time spent on corridors that every such path uses), and corridors per node (*static network indicators*). Results are in Table 5. Our three spine layouts span only 0.446–0.497 and also differ little in geometry, so "the indicators are too coarse" cannot be told apart from "the layouts are too similar". On the layouts of Lyu et al., changing only the machines on the same 4 × 4 grid (layout 2 to 3) changes the median share by 0.207, and three machine configurations on the same 5 × 5 grid span 0.075, whereas our three different networks span only 0.051. Meanwhile the LU cut is always 2 and the funnel share always 0 on the external batch (a station at a grid corner has two corridors, and the shortest paths to the machines share none), and corridors per node takes only two values. Because our side varies only the network, this does not show that machine configuration matters more than geometry, but that dimension cannot be ignored. **A constant indicator cannot explain a varying quantity: the static network indicators do not determine the impact-set size**; Section 7.1 points to schedule-dependent candidates.

6.5 E5: trade-off against global rescheduling

We test whether a *crossover* exists, i.e., whether R0+ overtakes bounded repair in $C_{\max}$ only once the budget is relaxed. R2 is a single deterministic computation run once per pair, so its $C_{\max}$ is the same at every budget; with the seed fixed, a larger budget lets R0+ run the same trajectory for more generations, so its best-so-far $C_{\max}$ is non-increasing. Hence, once the order is settled at the *smallest* budget, no larger budget can reverse it. At 0.2 s, R0+ is strictly better in 11 of the 12 pairs, ties in 1, and is worse in none; over all 36 budget points the win/tie/loss is 35/1/0 (both batches pooled; Table 6 and Figure 5 show the half on our layouts). No reversal at larger budgets follows from monotonicity for any instance; that R0+ is no worse at the smallest budget is only observed on these 12 pairs.

Below 0.2 s the order may reverse and was not tested: one decoding takes about 4.16 ms, so evaluating an initial population of 40 already takes about 0.17 s, whereas R2 needs only milliseconds. Neither R0+ nor R2 rewrites finished occupancies (0 at every budget point), so the makespan gap comes from global reassignment and resequencing. Bounded repair is two to three orders of magnitude faster, and both rewrite fractions are about 0.6. As Section 6.7 shows, this speed comes from single-pass rerouting, not from the impact set.

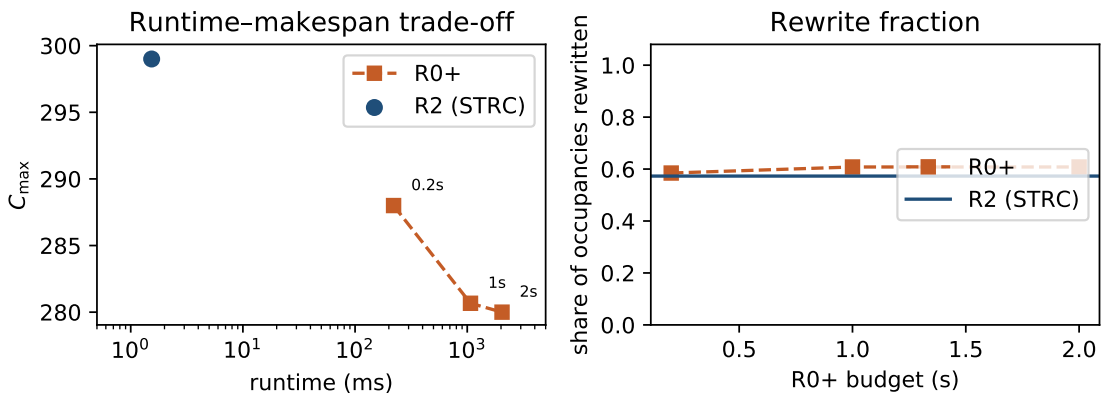


Fig. 5. E5: budget sweep of R0+ (which does not rewrite finished occupancies) versus bounded repair (congested instance, mean over 3 random seeds). Left: runtime versus C_{max}. Right: occupancy rewrite fraction.

Table 6. E5: warm-started global rescheduling R0+ versus bounded repair R2 (our layouts, mean over seeds {42, 7, 2024}). R2 runs once per pair and does not depend on the budget, so its values are identical in the three budget rows.

Instance	Budget/s	R0+ C_{max}	R2 C_{max}	Rewrite fraction R0+ / R2
Congested	0.2	288.0	299.0	0.58 / 0.57
	1.0	280.7	299.0	0.61 / 0.57
	2.0	280.0	299.0	0.61 / 0.57
Small	0.2–2.0	78.3	88.7	0.60 / 0.60

6.6 E6: disturbance types crossed with scope objects

On the same 50 pairs we test whether the conclusions of E1–E3 hold under corridor slowdown (traversal time doubled during the slowdown window), vehicle failure, and robotic-arm failure; the results are shown in Table 7. **First**, the A/B dichotomy holds without exception: scoping on G_T is empty for both Class B types and nonempty for both Class A types in every pair. Blockage and slowdown share the seed rule and hence give identical impact sets, so there are 150 independent scoping measurements, not 200. **Second**, over all 100 Class A measurements $U_{R1} \subseteq$ Cl, strictly in 87/100 pairs: switching to the reservation impact set never misses an occupancy already included by G_T. For robotic-arm failure this holds by construction (the seed rule takes the whole job suffix, while R1 truncates at $\theta = 2$); for vehicle failure it is an empirical result. The price is more rewritable occupancies on average: from 31.1 (scoping on G_T) to 60.6 (impact set) under vehicle failure, and from 42.8 to 65.9 under robotic-arm failure. **Third**, all four types are repaired in every pair without escalation, Class A by vehicle swapping or machine reassignment. **Fourth**, under blockage the impact set contains about 92% of the unfinished occupancies, leaving out fewer than 10%; the Cl/$|R|$ of 0.56–0.59 in Table 4 has all occupancies as denominator, about two fifths of which are finished before t_{now}. These results extend the conclusions on C1 and C2 from corridor blockage to four disturbance types.

Table 7. E6: disturbance type × scope object (50 pairs per type). $|R^\circ|$: unfinished occupancies. "R1 empty": pairs where scoping on G_T deems no rescheduling necessary. The last column uses no scope escalation.

Disturbance	Class	R1 empty	Mean $\lvert U_{R1}\rvert$	Mean $\lvert\mathrm{Cl}\rvert$	$\mathrm{Cl}/\lvert R^\circ\rvert$	R2 feasible
Corridor blockage	B	50/50	0.0	64.4	0.92	50/50
Corridor slowdown	B	50/50	0.0	64.4	0.92	50/50
Vehicle failure	A	0/50	31.1	60.6	0.85	50/50
Robotic-arm failure	A	0/50	42.8	65.9	0.94	50/50

Table 8. E7: methods within the millisecond range (50 pairs, same setup as E3, all methods feasible in every pair). "Violations": pairs with rewritten finished occupancies; "Rewritable": share of unfinished occupancies included in the rewrite; last column: $C_{\max}$ win/tie/loss against R2.

Method	Violations	ms (median)	Mean degradation	Rewrite fraction	Rewritable	W/T/L vs. R2
RS right shift	0/50	1.42	58.8%	0.627	—	7/16/27
RD re-decoding	0/50	4.16	49.1%	0.618	—	22/14/14
RA all unfinished	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 bounded repair	0/50	2.74	49.8%	0.540	0.92	—

6.7 E7: comparisons within the millisecond range

E5 leaves open how much of the speed advantage is due to the impact set and how much merely to a slow baseline. We therefore compare R2 with three millisecond baselines (Table 3). **RS** postpones all unfinished occupancies by the smallest delay that clears the blocked corridor; because the shift is uniform, all "≥" constraints remain satisfied without rerouting. **RD** keeps the chromosome and runs one fixed-prefix decoding, the fastest global recomputation in our implementation. **RA** makes all unfinished occupancies rewritable and runs the *identical* replay as R2; it is the only baseline that isolates the effect of the impact set.

RS needs 1.42 ms, about half of R2, and is the only feasible method faster than R2 (R1 is faster still, but feasible in only 0/50 pairs); it degrades the makespan by 58.8% versus 49.8% (R2 win/tie/loss 27/16/7) and rewrites more (0.627 vs. 0.540). Making the shift selective would first require determining which occupancies are affected—precisely the scope delineated here. **The right shift is fast because it gives up selectivity.**

RD takes 4.16 ms, 1.5 times R2, and any decoding-based search needs at least one decoding per candidate. On makespan, R2 and RD are indistinguishable (14/14/22; mean paired difference +0.006, CI [−0.007, 0.019]; Wilcoxon $p = 0.12$), while RD rewrites more (0.618). **Even the fastest global recomputation is slower, so the speed gap in E5 is not an artifact of a slow baseline.**

On average the impact set excludes only $1 - 0.92$ of the unfinished occupancies (0.80–1.00 per pair; in 6 pairs the two sets coincide). This small difference lowers the rewrite fraction from 0.604 to 0.540 (mean reduction 0.064, median 0.029, per pair 31/18/1 for R2 fewer/equal/more rewrites, $p = 8.8 \times 10^{-7}$; Hodges–Lehmann estimate 0.037, CI [0.018, 0.074]; CI of the mean [0.039, 0.093]; excluding the 6 pairs with identical results, 31/12/1 with mean 0.073), while runtimes (2.74 vs. 2.76 ms) and makespans (mean ratio 0.999; R2 never worse) are nearly identical. **Whether scoping reduces rewriting can be answered only by R2 versus RA: R2 rewrites less.** The reduction grows as the impact set excludes more: 0.010 on the congested instance (rewritable share 0.94) versus 0.118 on the mid layout (0.91). Figure 6 condenses this into a post hoc rule: the 23 pairs with $\mathrm{Cl}/|R^\circ| \geq 0.93$ have a mean reduction of 0.029, and the 27 pairs below it 0.094;

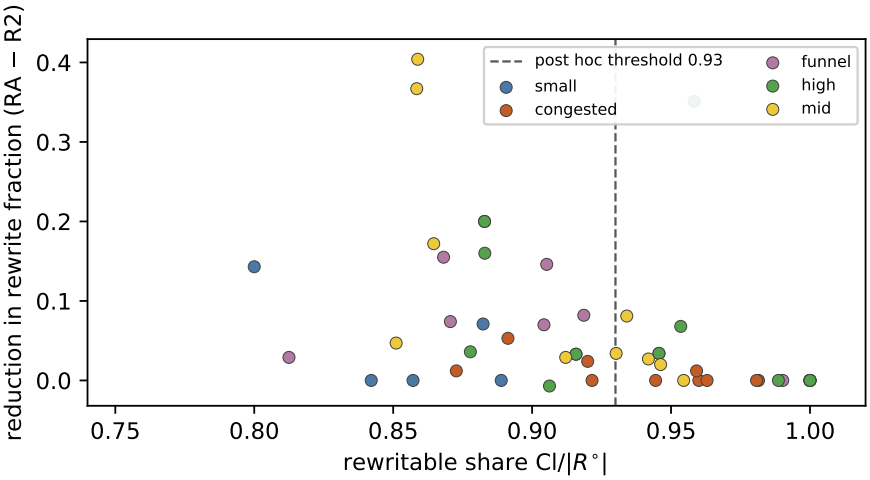


Fig. 6. E7 post hoc rule: rewritable share $\mathrm{Cl}/|R^{\circ}|$ versus the reduction in rewrite fraction of R2 relative to RA (50 pairs). The dashed line marks the in-sample threshold 0.93.

above 0.97 the reduction is zero (necessarily so in the 6 pairs with share 1.00, where R2 and RA coincide; empirically in the rest). The threshold is in-sample and not meant for extrapolation. Taken together, equal runtime shows that the millisecond response comes from single-pass rerouting, not from the impact set; equal makespan shows that the impact set is not an optimization device; and R2 rewrites less because occupancies outside the impact set keep the original plan by Proposition 3. **The benefit of the impact set lies in rewrite volume, not in speed or makespan**: at equal runtime and makespan, the share of rewritten occupancies is 0.604 for RA and 0.540 for R2, about 6 percentage points apart; RA rewrites about 12% more occupancies.

6.8 Sweep over disturbance magnitude

We test whether bounded repair stays feasible and fast as the disturbance grows: on the congested and the small instance (10 seeds each), the share φ of disturbed operations after t_{now} takes 0.10, 0.25, 0.50, 0.75, and 1.00; unlike in E3 and E5, escalation is enabled, and R0+ has a 2 s budget. Table 9 shows that bounded repair is feasible in every pair at every level (also on the small instance); its level means stay at 8.4–11.1 ms (2.2–23.8 ms per pair; about 1 ms on the small instance) with no upward trend, including 10.7 ms at $\varphi = 1$. R0+ attains the better $C_{\max}$ at every level, as in E5, but uses its full 2 s budget, about two orders of magnitude more time. **Within the tested range, bounded repair remains feasible and its runtime stays on the order of ten milliseconds, without growing with the disturbance magnitude.**

6.9 E8: extrapolation to larger instances

We test whether the impact-set share approaches 1 as the instance grows, which would make "bounded" meaningless. We use $2k$ jobs, k machines, and k vehicles for $k = 4, \ldots, 10$ (7 levels × 10 seeds = 70 pairs), with congestion, heterogeneity, flexibility, time ratio, and station cuts calibrated to fixed targets, so that only size changes; every level is generated from instance seed 42 at one congestion level.

Table 9. Magnitude sweep (congested instance, means over 10 seeds; escalation enabled; R0+ budget 2 s).

φ	Bounded repair feasible	Bounded repair ms	R0+ $C_{\max}$
0.10	100%	9.1	103.5
0.25	100%	11.1	107.0
0.50	100%	8.6	111.8
0.75	100%	8.4	125.8
1.00	100%	10.7	148.1

Table 10. E8: instance-size sweep (7 levels × 10 seeds). "Impact set": Cl/|R|, whose attainable upper bound is $|R^{\circ}|/|R| \approx 0.61$. Runtimes are medians; "RD/R2" is the ratio of the two median columns; the last column is the $C_{\max}$ win/tie/loss of R2 against RS.

Size	$\lvert R\rvert$	Impact set	R2 ms	RD ms	RD/R2	R2 vs. RS
$8 \times 4 \times 4$	148	0.561	2.9	4.4	1.5×	7/1/2
$10 \times 5 \times 5$	172	0.485	3.2	6.1	1.9×	9/0/1
$12 \times 6 \times 6$	230	0.481	4.5	10.7	2.4×	7/1/2
$14 \times 7 \times 7$	281	0.492	6.1	14.5	2.4×	8/0/2
$16 \times 8 \times 8$	291	0.534	6.4	16.4	2.6×	10/0/0
$18 \times 9 \times 9$	356	0.535	8.2	26.4	3.2×	10/0/0
$20 \times 10 \times 10$	402	0.476	10.4	34.2	3.3×	10/0/0

Table 10 shows that Cl/|R| fluctuates between 0.48 and 0.56 without trend; since unfinished occupancies make up only about 0.61 of the table, this share cannot approach 1, and relative to the unfinished occupancies it lies in 0.78–0.91 (0.91 at the smallest size, 0.78 at the largest), the same order as the 92% of the main batch. As |R| grows 2.7-fold, the median runtime of R2 grows from 2.9 to 10.4 ms; the first round of rerouting (without escalation) is feasible in 69/70 pairs, and the remaining pair succeeds in escalation round 3 (12.0 ms), giving 70/70. The RD/R2 ratio rises from 1.5× to 3.3×, not monotonically but by far more than any within-level IQR (at most 0.74). Against RS, R2 wins 7–10 of 10 pairs per level, with no trend. Seven levels do not support a claim about larger sizes, and the impact set is not small; but **across the seven levels, the impact-set share does not rise with size, R2 stays on the order of ten milliseconds, and its runtime advantage over global recomputation widens**.

6.10 Reproduction on the external-layout batch

We test whether E1, E2, E3, and E5 depend on our layouts by rerunning them on layouts 2–6 of Lyu et al. (50 pairs; E5 on layouts 2 and 6 with seeds $\{42, 7, 2024\}$ and budgets $\{0.2, 1, 2\}$ s, 18 budget points). The scope on G_T is empty in 50/50, closure holds in 50/50, fields outside the set are unchanged in 50/50, and single-pass rerouting is feasible in 0/50 pairs when scoping on G_T and in 50/50 when scoping on the impact set. The per-layout median share lies in 0.438–0.547 (main batch: pooled median 0.571). R0+ attains the better $C_{\max}$ at 17 of the 18 budget points and ties at the remaining one, with no crossover. **Hence the conclusions on the scope object (C1, C2) do not depend on our layouts.**

7 Conclusion

When a flexible job shop records corridor occupancies with time windows, the operation-precedence graph contains no yielding relations between occupancies and cannot capture disturbances that invalidate only occupancies: under corridor blockage, operation-level scoping is empty although the original schedule is infeasible (all 50 pairs, C1). Delineating the scope on the reservation dependency graph makes the occupancies to be rewritten uniquely determined by the

given relations rather than by a heuristic radius (closure and outside invariance in all 50 pairs, C2), and with the same rerouting and only the scope object changed, feasibility goes from 0/50 to 50/50 (C3). Rewriting all unfinished occupancies instead leaves runtime and makespan almost unchanged but raises the rewrite fraction from 0.540 to 0.604; this difference shrinks as the impact set approaches the set of all unfinished occupancies. Hence the millisecond response comes from single-pass rerouting, and **scoping on the impact set serves to reduce rewriting: occupancies outside it are kept as planned** (Proposition 3). Yielding relations, rerouting inside the set, and scope escalation are taken from existing practice.

7.1 Limitations and future work

We list four limitations in order of their impact, each followed by the corresponding direction.

(1) **Makespan.** Warm-started global rescheduling is no worse than bounded repair on $C_{\max}$ at every tested budget, with no crossover (Section 6.5). Since neither method rewrites finished occupancies, narrowing the gap would require limited reassignment or resequencing, which drifts toward global rescheduling; this limitation concerns the benefit of scoping, not the conclusions on the scope object (C1, C2).

(2) **Guarantee and size of the impact set.** Proposition 3 is only a sufficient condition, and the edge set has not been verified exhaustively. The impact set is not the minimal set needed to restore feasibility: it covers about 92% of the unfinished occupancies, so the reduction in rewriting is limited. The probe finds 0 leaks for the known abutting omission; other disturbance forms (shortened occupancies, longer delays) should be probed next. A minimal rewritable set defined for a fixed repair capability would quantify the gap. Static network indicators cannot predict the impact-set size (Section 6.4); the next candidates are schedule-dependent, e.g., the number of overlapping occupancies on the disturbed corridor near t_{now}. This limitation does not affect the conclusion of C1.

(3) **Scope of the implementation.** Class B is handled by rerouting only, Class A adds vehicle swapping and reassignment without resequencing, and only a single disturbance is handled (A3). Size has been extended to 20 jobs and 10 machines, but E8 has only seven levels, too few to infer the trend at larger sizes. The next step is to relax A3 and examine whether the impact set grows cumulatively under continual disturbances.

(4) **External reference points.** No public benchmark can be substituted in: the public instance families [2, 3] publish machine-to-machine travel-time matrices; none of their 11 layouts can be reconstructed as a corridor graph consistent with the matrix, so R, G_R, $\mathrm{Cl}(\cdot)$, and Class B disturbances cannot be defined on them. Our second batch takes only the layouts from Lyu et al. [9]; travel times and job data are still set by us, so the reference values of Lyu et al. or van Os [23] cannot be reproduced. An instance family publishing both a network and per-leg travel times is still needed; our edge-weight sweep in E3 tests only robustness and cannot replace it. Hence only the networks come from the literature and external validity is limited; relative comparisons within the same implementation are unaffected.

References

[1] Andrea D'Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034

[2] Dalila B. M. M. Fontes, S. Mahdi Homayouni, and José F. Gonçalves. 2023. A hybrid particle swarm optimization and simulated annealing algorithm for the job shop scheduling problem with transport resources. *European Journal of Operational Research* 306, 3 (2023), 1140–1157. doi:10.1016/j.ejor.2022.09.006

[3] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[4] Ahmad Reza Jafarian-Moghaddam, Ali Esmailpoor Sanjabi, and Elham Mortazavi Moghaddam. 2026. Integrated tri-objective scheduling and internal logistics optimization in U-shaped flexible flow shops: a mathematical model and metaheuristic framework. *International Journal of Systems Science: Operations & Logistics* 13, 1 (2026), 2714000. doi:10.1080/23302674.2026.2714000

[5] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[6] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, Honolulu, HI, USA, 79–86. doi:10.1109/SCIS.2007.367673

[7] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[8] Sirui Li, Wenbin Ouyang, Yining Ma, and Cathy Wu. 2025. Learning-guided rolling horizon optimization for long-horizon flexible job-shop scheduling. arXiv:2502.15791 [cs.LG] doi:10.48550/arXiv.2502.15791

[9] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[10] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Abu Dhabi, United Arab Emirates, 10214–10221. doi:10.1109/IROS58592.2024.10801975

[11] François Margot, Maurice Queyranne, and Yaoguang Wang. 2003. Decompositions, network flows, and a precedence constrained single-machine scheduling problem. *Operations Research* 51, 6 (2003), 981–992. doi:10.1287/opre.51.6.981.24912

[12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1

[13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8

[15] Fei Qiao, Li Li, Yumin Ma, and Bin Shi. 2012. Single Machine Oriented Match-Up Rescheduling Method for Semiconductor Manufacturing System. In *Intelligent Robotics and Applications (ICIRA 2012) (Lecture Notes in Computer Science, Vol. 7507)*. Springer, Berlin, Heidelberg, 217–226. doi:10.1007/978-3-642-33515-0_22

[16] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[17] Miguel A. Salido, Joan Escamilla, Federico Barber, and Adriana Giret. 2017. Rescheduling in job-shop problems for sustainable manufacturing systems. *Journal of Cleaner Production* 162 (2017), S121–S132. doi:10.1016/j.jclepro.2016.11.002

[18] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6

[19] Yong Jin Suh and Jin Young Choi. 2022. Efficient Fab Facility Layout with Spine Structure Using Genetic Algorithm under Various Material-Handling Considerations. *International Journal of Production Research* 60, 9 (2022), 2816–2829. doi:10.1080/00207543.2021.1904159

[20] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[21] Jiří Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. AAAI Press, Honolulu, HI, USA, 7732–7739. doi:10.1609/aaai.v33i01.33017732

[22] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003

[23] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[24] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958

[25] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[26] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103

[27] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, Jeju, South Korea, 311–320. doi:10.24963/ijcai.2024/35